ONLINE PARAMETER ESTIMATION OF A DC MOTOR USING RECURSIVE LEAST SQUARES

1.0 Learning Outcomes.....1
2.0 Hardware Set-Up.....1
3.0 Practical Investigation via Simulink.....2
4.0 Concluding Remarks.....6
Recommended Citation.....6

1.0 Learning Outcomes

Upon completing this practical exercise, students should be able to demonstrate the Learning Outcomes presented in Table 1. At the end of the exercise, revisit each Learning Outcome and confirm that it has been achieved.

Table 1: *Learning Outcomes for the Practical Exercise*

ID	Learning Outcome	Achieved
1	Run a recursive least squares (RLS) estimator in real time on the ACE-Lab, and observe the gain, time constant and dead-band of a DC motor being estimated while the motor runs.	☐
2	Compare the final RLS estimate with a batch linear least squares (LLS) estimate calculated from the same logged data, and validate the identified model against the measured response.	☐
3	Investigate how the forgetting factor affects the speed of convergence and the variability of the estimate, and choose a suitable value for an adaptive controller.	☐

2.0 Hardware Set-Up

This practical exercise builds on the hardware and algorithm set-up described in the practical exercises, DC Motor Speed Measurement using an Encoder, and Parameter Estimation of a DC Motor using Linear Least Squares, which detail the operation of a DC motor with an encoder and the identification of a first-order model. The set-up measures the speed of the motor in revolutions per minute (RPM) and is not repeated here.

The ACE-Lab (Base + Sense) and ACE-Lab (Actuate) are used side by side, wired as in Figure 1. The model runs on an Arduino Mega 2560 at 100 Hz in external mode (Monitor & Tune), so that every signal can be watched and logged while the motor runs; the Arduino Mega 2560 uses the same pins as the Arduino in the ACE-Lab (Base + Sense) (5V, GND, D2, D3 and D5). The 1:298 geared 6V DC motor is driven through the Cytron Maker Drive from a 9V battery, and its speed is measured with a quadrature encoder giving 3576 counts per output-shaft revolution.

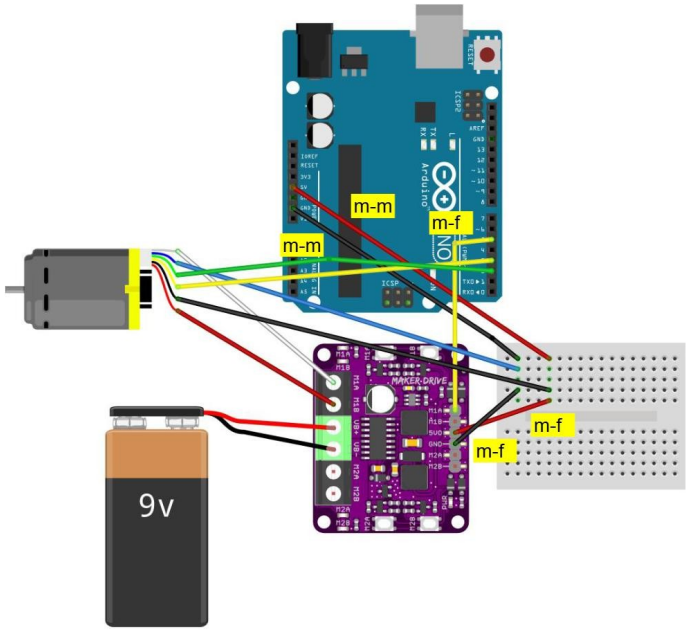


Figure 1: *Hardware Set-Up for DC Motor Speed Measurement and Control. The Figure Shows the Arduino from the ACE-Lab (Base + Sense); the Arduino Mega 2560 Uses the Same Pins*

3.0 Practical Investigation via Simulink

MATLAB and Simulink should be used to complete this practical exercise. In the practical exercise, Parameter Estimation of a DC Motor using Linear Least Squares, the data were logged first and the model was identified afterwards. Here, the model is identified while the motor is running, using RLS, as described in the Recursive Least Squares for Online Parameter Estimation Learning Guide. A known, changing voltage is applied to the motor in open loop, and the gain, time constant and dead-band of the motor are updated every 0.05 s and displayed live.

The Simulink Model

Open `dc_motor_rls.slx`. The model contains the following parts:

- **Open-Loop Excitation:** a constant PWM duty of 150 plus two square waves of amplitude 15 and 5, with periods of 3.0 s and 7.4 s, starting at t = 1 s. The duty varies between 130 and 170, which with the 9V supply is 4.6 V to 6 V. This keeps the motor above its sticking region while changing the input often enough to excite its dynamics.
- **Speed Measurement:** the encoder count is converted into RPM, as in the practical exercise, DC Motor Speed Measurement using an Encoder, and smoothed with the low-pass filter $0.2/(1-0.8z^{-1})$.
- **Input Filter:** the applied voltage is passed through the same low-pass filter as the speed, so that the lag of the filter does not bias the estimated time constant.
- **RLS Estimator:** a MATLAB Function block that estimates the model $y_k = -a_1 y_{k-1} + b_0 u_{k-1} + c$ every D = 5 samples (T_e = 0.05 s), using the average voltage over each interval. The constant c accounts for the dead-band of the motor.
- **Displays, Scopes and logging:** the estimate $\hat{\theta} = [a_1 \ b_0 \ c]^T$, the gain K (RPM/V) and the time constant τ (s) are shown live, and the voltage (`u_data`), filtered speed (`y_data`) and estimate (`theta_est`) are logged to the workspace.

The parameters of the model are stored in its model workspace (Modeling > Model Explorer > Model Workspace), see Table 2.

Table 2: *Parameters of dc_motor_rls.slx*

Parameter	Value	Meaning
`Ts`	0.01 s	Sample time of the model
`T_exp`	60 s	Length of the experiment
`pwm_offset`, `pwm_amp1`, `pwm_amp2`	150, 15, 5	Excitation: PWM duty 130 to 170
`per1`, `per2`	3.0 s, 7.4 s	Periods of the two square waves
`filt_num`, `filt_den`	0.2, [1 −0.8]	Speed (and voltage) low-pass filter
`D`	5	Estimate every 5 samples (T_e = 0.05 s)
`lambda`	0.995	Forgetting factor (memory about 10 s)
`P0`, `TraceMax`	1000, 10^4	Initial covariance, wind-up limit

The RLS Estimator

Double-click the RLS Estimator block to see its code. The update below is the core of the block, and follows Equations (5), (7) and (10) of the Recursive Least Squares for Online Parameter Estimation Learning Guide. The estimate starts from zero with a large covariance, so the first few seconds of data correct it quickly.

MATLAB Function: the estimation step of the RLS Estimator block

```
count = count + 1;
if count >= D                           % every D samples (Te = 0.05 s)
    uAvg = uSum/D;                      % average input over the interval
    phi = [-yPrev; uAvg; 1];            % regressor
    if (k - D)*Ts >= t_start            % only use data after the start
        e = y - phi'*th;                % prediction error
        Pphi = P*phi;
        lam = lambda;
        if trace(P) > TraceMax          % anti wind-up
            lam = 1;
        end
        L = Pphi/(lam + phi'*Pphi);     % gain vector
        th = th + L*e;                  % update the estimate
        P = (P - L*Pphi')/lam;          % update the covariance
        P = (P + P')/2;                 % keep P symmetric
    end
    yPrev = y;  uSum = u;  count = 0;   % start the next interval
else
    uSum = uSum + u;
end
```

Run the Estimator on the Hardware

1. Connect the Arduino Mega 2560 and check that the battery is connected to the Cytron Maker Drive. In the Hardware tab, check that the hardware board is Arduino Mega 2560.
2. Open the Voltage and Speed and the Estimates scopes, then press **Monitor & Tune**. The motor runs for 60 s.
3. Watch the K (RPM per V) and tau (s) displays. Record the estimates at t = 5 s, 10 s, 30 s and 60 s in Table 4.

4. When the run has finished, plot the estimates against time using the script below, and record the final values of K, τ and the dead-band, $V_{dead} = -c/b_0$.

MATLAB Script: plotting the logged RLS estimates (`plot_estimates.m`)

```
Ts = 0.01;  D = 5;  Te = D*Ts;
u  = double(out.u_data(:));              % voltage [V]
y  = double(out.y_data(:));              % filtered speed [RPM]
th = double(out.theta_est);              % estimates [a1 b0 c]
if ndims(th) == 3, th = reshape(th, 3, [])';
elseif size(th, 2) ~= 3, th = th'; end
t  = (0:numel(y)-1)'*Ts;
a1 = th(:,1);  b0 = th(:,2);  c = th(:,3);
K  = b0./(1 + a1);                       % gain [RPM/V]
Tau = -Te./log(-a1);  Tau(-a1 <= 0 | -a1 >= 1) = NaN;   % time constant [s]
fprintf('K = %.2f RPM/V, tau = %.3f s, dead-band = %.2f V\n', ...
        K(end), Tau(end), -c(end)/b0(end));
subplot(311); yyaxis left; plot(t, y); ylabel('Speed (RPM)');
yyaxis right; stairs(t, u); ylabel('Voltage (V)'); grid on;
subplot(312); plot(t, K); grid on; ylabel('K (RPM/V)');
subplot(313); plot(t, Tau); grid on; ylabel('\tau (s)'); xlabel('Time (s)');
```

Example Results

Before using the hardware, the same model can be run without it: `dc_motor_rls_virtual.slx` replaces the Arduino blocks with a virtual motor (K = 16 RPM/V, τ = 0.25 s and a 0.5 V dead-band, supplied from 6 V). Example results from the virtual model are shown in Figure 2. The estimates approach the true values within about 10 to 20 s, and after 60 s the RLS estimate is K = 15.98 RPM/V, τ = 0.249 s and V_{dead} = 0.50 V, almost exactly the values of the virtual motor; the small difference is due to the quantisation of the encoder.

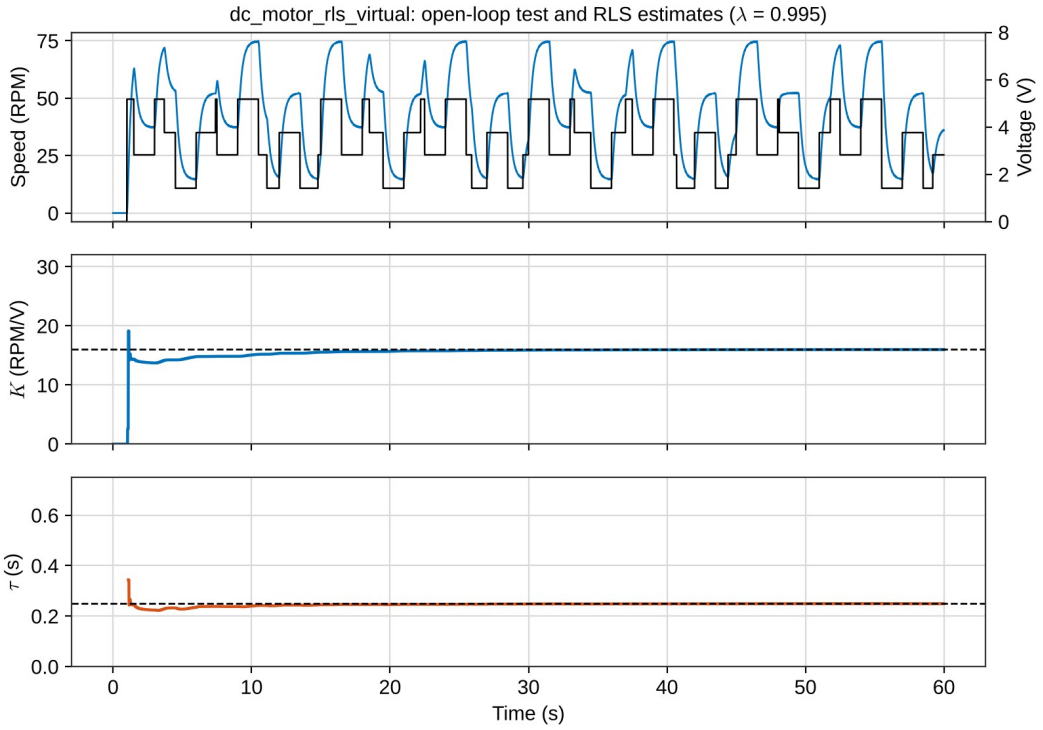


Figure 2: *Example Results from dc_motor_rls_virtual.slx: the Open-Loop Test (Top), and the Estimated Gain (Middle) and Time Constant (Bottom); the Dashed Lines Show the Values of the Virtual Motor*

On the hardware, the speed is noisier and the motor is faster. For comparison, a batch LLS identification of an ACE-Lab motor with the same excitation gave K =

15.99 RPM/V, τ = 0.054 s and a dead-band of 0.20 V, with a fit of 66.5%, see Figure 3. Your RLS estimate should be similar, but each motor is slightly different.

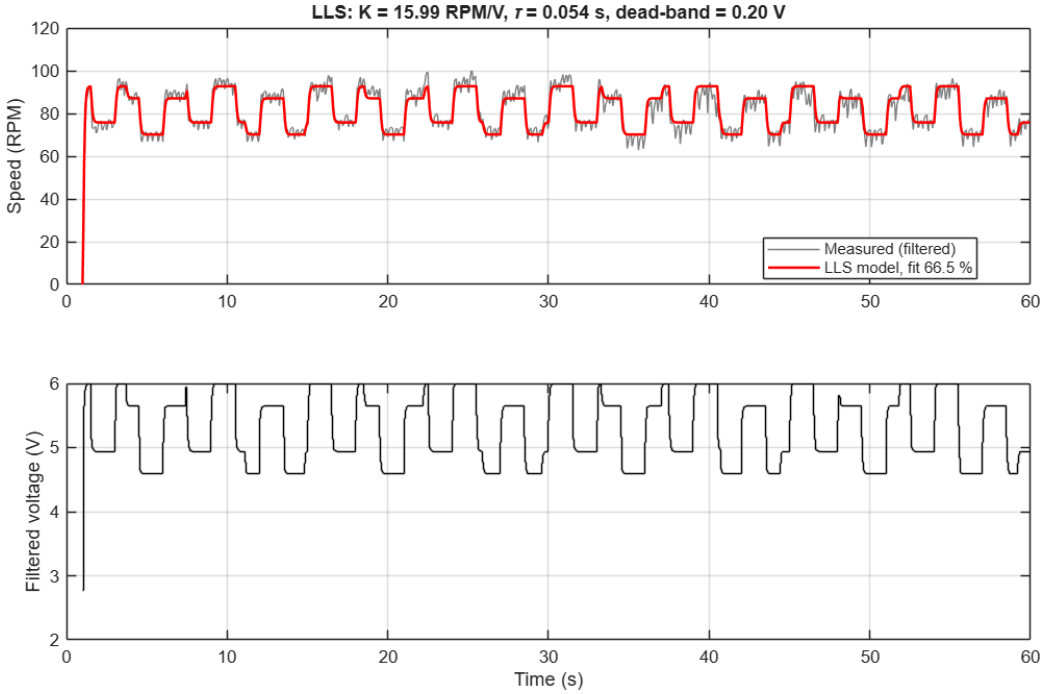


Figure 3: *Batch LLS Identification of an ACE-Lab Motor from a 60 s Open-Loop Test: Measured and Simulated Speed (Top) and Filtered Voltage (Bottom)*

Compare the RLS and LLS Estimates

With λ = 1, RLS gives the same result as batch LLS. With λ = 0.995, the RLS estimate is weighted towards the last 10 s or so of data, so the two estimates should be close but not identical.

1. After the hardware run, run `lls_offline.m`. It calculates the batch LLS estimate of a_1, b_0 and c from the same logged data (`u_data` and `y_data`), simulates the identified model from the measured voltage, and reports the fit.
2. Record the LLS and RLS estimates in Table 3, and compare them. Is the fit of your model similar to the 66.5% in Figure 3? Explain why the fit on the hardware is much lower than in the virtual model.

Table 3: *RLS and LLS Estimates for Your Motor*

Method	K (RPM/V)	τ (s)	V_{dead} (V)	Fit (%)
RLS, λ = 0.995 (final value)				–
LLS, `lls_offline.m`				

Investigate the Forgetting Factor

The forgetting factor sets the memory of the estimator, about $T_e/(1-\lambda)$ seconds. A short memory follows changes quickly, but the estimate is more affected by noise.

1. Change `lambda` in the model workspace to 1, then repeat the hardware run. Record the estimates at 5 s, 10 s, 30 s and 60 s in Table 4.
2. Repeat with λ = 0.98 and λ = 0.95.
3. For each run, calculate the standard deviation of the estimated gain over the last 20 s, using `std(K(t > 40))`, and record it in Table 4.

4. Plot the four estimates of K on the same axes. Which value of λ would you choose for an adaptive controller that must follow a sudden change in load within about 10 s, and why?

Table 4: *Effect of the Forgetting Factor on Your Motor*

λ	Memory (s)	K at 5 s	K at 10 s	K at 30 s	K at 60 s	τ at 60 s	Std. dev. of K, 40–60 s
1	All data						
0.995	10						
0.98	2.5						
0.95	1						

With $\lambda = 1$ the estimate becomes very steady, but it would take a long time to respond to a change in the motor. With $\lambda = 0.95$ (memory of 1 s) the estimate follows every fluctuation in the noisy speed measurement: at 0.01 s, one encoder count corresponds to about 1.68 RPM. Explain the differences in your own results, and relate them to the trade-off described in the Recursive Least Squares for Online Parameter Estimation Learning Guide.

4.0 Concluding Remarks

This practical exercise identified a model of a DC motor while the motor was running. Using MATLAB, Simulink and ACE-Lab hardware, a recursive least squares estimator, running every 0.05 s on the Arduino Mega 2560, updated the gain, time constant and dead-band of the motor from the applied voltage and the measured speed, without storing the data or inverting a matrix. The final estimate was compared with a batch LLS estimate from the same data, and validated against the measured response.

This practical exercise also showed that the forgetting factor sets a trade-off between how quickly the estimate can follow a change and how steady it is in the presence of noise. An estimator that runs continuously, with a memory of a few seconds, provides an up-to-date model of the motor that can be used to recalculate the gains of a controller automatically, as in the practical exercise, DC Motor Self-Tuning PI Control using Recursive Least Squares.

Recommended Citation

If you have used this practical exercise in your teaching, learning or research, please cite it using the reference below, adapting the format as required by your preferred referencing style.

Pickering, J. E. (2026). Online Parameter Estimation of a DC Motor using Recursive Least Squares. ACE-Lab. Teaching Notes: ACE-Lab Practical Exercises, Version 1.0. Zenodo. https://doi.org/10.5281/zenodo.23671549